



COMPTE RENDU

Techniques de réduction de dimensions et de visualisation

Élève :

Aymen RGUEG

Professeur :

Oumaima BANOVAR

17 novembre 2025

Table des matières

1	Introduction :	4
2	Chargement des données :	4
3	Interprétation des données :	6
4	Standardisation :	7
5	Les valeurs propres :	11
6	Détermination des axes à conserver pour le k-means sur les performances des athlètes en dimension réduite	13
7	Représentation graphique des individus et des sports :	15
8	Travail en autonomie :	16
9	Conclusion :	17

Abstract :

This report explores the application of Principal Component Analysis (PCA) to a comprehensive dataset of decathlon athletes' performances from the 2004 Olympic Games and the Décastar competition. Our goal is to distill the complexity of the data while retaining critical information that enhances exploratory analysis. By examining the correlations among the ten distinct events, we aim to categorize athletes based on their unique performance profiles. Through eigenvalue analysis, we identify the principal components that capture significant variance, determining the optimal number of dimensions for effective interpretation. Visual representations, such as biplots and scree plots, further illuminate these findings, showcasing individual trajectories and highlighting performance clusters. The insights gleaned reveal compelling factors that differentiate top competitors and serve as predictive indicators for future competitions. Ultimately, this study demonstrates the power of PCA in transforming intricate athletic data into actionable insights, enriching our understanding of performance dynamics in decathlon and paving the way for athletes to maximize their potential.

Résumé :

Ce rapport présente une analyse approfondie utilisant l'Analyse en Composantes Principales (ACP) sur les performances des athlètes participants au décathlon lors des Jeux Olympiques de 2004 et de la compétition Décastar. L'objectif principal de cette étude est de simplifier la complexité des données tout en préservant les éléments cruciaux nécessaires à une analyse exploratoire efficace. Nous examinons les interrelations entre les différentes épreuves du décathlon pour comprendre comment ces corrélations peuvent être utilisées pour classer les athlètes selon leurs performances respectives. Grâce à une analyse des valeurs propres et à des visualisations graphiques percutantes, nous identifions le nombre optimal de dimensions requises pour interpréter les données de manière significative. Les résultats obtenus mettent en lumière des schémas distincts qui non seulement différencient les vainqueurs, mais offrent également des indications précieuses pour prédire les résultats des compétitions à venir.

1 Introduction :

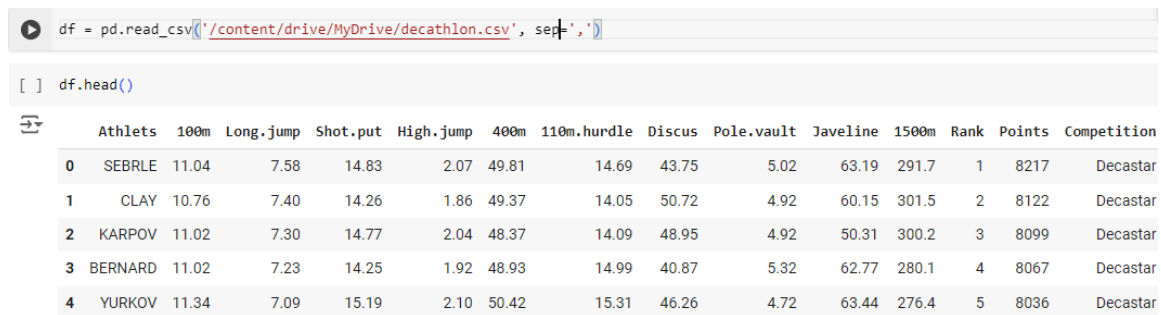
L'analyse des données complexes est cruciale pour extraire des informations pertinentes, notamment lorsqu'il s'agit de grands ensembles de données. L'Analyse en Composantes Principales (ACP) est une technique statistique permettant de réduire la dimensionnalité des données tout en préservant au maximum les variations. Ce rapport présente une étude de cas appliquée aux performances des athlètes de décathlon des Jeux Olympiques et du Décastar 2004, visant à extraire des informations significatives à l'aide de l'ACP. L'objectif est de comprendre comment réduire le nombre de variables tout en conservant la structure des données, et d'explorer les caractéristiques distinctives des athlètes.

2 Chargement des données :

Le chargement des données est une étape essentielle qui précède toute analyse, car il permet de préparer et d'exploiter les informations disponibles. Dans le cadre de ce projet, nous utilisons un ensemble de données concernant les performances des athlètes de décathlon lors des Jeux Olympiques et du Décastar 2004, accessible au format CSV sur Kaggle. Pour importer ces données, nous avons recours à plusieurs bibliothèques Python : Pandas pour la manipulation des données, Numpy pour les calculs mathématiques, Matplotlib et Seaborn pour la visualisation, ainsi que Scikit-learn pour effectuer l'Analyse en Composantes Principales (ACP). Le fichier CSV est chargé dans un DataFrame grâce à la fonction `pd.read_csv` de Pandas, ce qui simplifie considérablement le processus d'analyse.

Le chargement des données est une étape essentielle qui précède toute analyse, car il permet de préparer et d'exploiter les informations disponibles. Dans le cadre de ce projet, nous utilisons un ensemble de données concernant les performances des athlètes de décathlon lors des Jeux Olympiques et du Décastar 2004, accessible au format CSV sur Kaggle. Pour importer ces données, nous avons recours à plusieurs bibliothèques Python : Pandas pour la manipulation des données, Numpy pour les calculs mathématiques, Matplotlib et Seaborn pour la visualisation, ainsi que Scikit-learn pour effectuer l'Analyse en Composantes Principales (ACP). Le fichier CSV est chargé dans un DataFrame grâce à la fonction `pd.read_csv` de Pandas, ce qui simplifie considérablement le processus d'analyse.

Une fois les données chargées, il est crucial d'effectuer un nettoyage et une exploration préliminaire pour s'assurer de leur qualité. Cela inclut la vérification des valeurs manquantes, des doublons et des incohérences qui pourraient fausser les résultats. En analysant les premières lignes du DataFrame, on peut également obtenir un aperçu des types de données et de leur distribution, ce qui est fondamental pour choisir les méthodes statistiques appropriées par la suite. Par ailleurs, cette étape préparatoire permet d'identifier des variables potentiellement pertinentes pour l'analyse, facilitant ainsi l'application de techniques avancées comme l'ACP pour révéler des insights significatifs sur les performances des athlètes.



```
df = pd.read_csv('/content/drive/MyDrive/deathlon.csv', sep=',')
```

```
[ ] df.head()
```

	Athlets	100m	Long.jump	Shot.put	High.jump	400m	110m.hurdle	Discus	Pole.vault	Javeline	1500m	Rank	Points	Competition
0	SEBRLE	11.04	7.58	14.83	2.07	49.81	14.69	43.75	5.02	63.19	291.7	1	8217	Decastar
1	CLAY	10.76	7.40	14.26	1.86	49.37	14.05	50.72	4.92	60.15	301.5	2	8122	Decastar
2	KARPOV	11.02	7.30	14.77	2.04	48.37	14.09	48.95	4.92	50.31	300.2	3	8099	Decastar
3	BERNARD	11.02	7.23	14.25	1.92	48.93	14.99	40.87	5.32	62.77	280.1	4	8067	Decastar
4	YURKOV	11.34	7.09	15.19	2.10	50.42	15.31	46.26	4.72	63.44	276.4	5	8036	Decastar

FIGURE 1 – Chargement du fichier CSV et affichage des premières lignes du dataset

Ce code utilise la bibliothèque Pandas pour lire un fichier CSV intitulé "deathlon.csv" et le convertir en un DataFrame. La fonction `pd.read_csv('deathlon.csv', sep=',')` charge le fichier CSV en indiquant que les colonnes sont séparées par des virgules grâce au paramètre `sep=','`. Les données importées sont stockées dans la variable `df`, qui représente une structure de données bidimensionnelle. Par la suite, la méthode `df.head()` est utilisée pour afficher les cinq premières lignes du DataFrame, fournissant ainsi un aperçu rapide de la structure et du contenu des données importées.

3 Interprétation des données :

```
[ ] df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41 entries, 0 to 40
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  ---                ---
0   Athlets                41 non-null    object
1   100m                   41 non-null    float64
2   Long.jump              41 non-null    float64
3   Shot.put               41 non-null    float64
4   High.jump              41 non-null    float64
5   400m                   41 non-null    float64
6   110m.hurdle            41 non-null    float64
7   Discus                 41 non-null    float64
8   Pole.vault             41 non-null    float64
9   Javeline               41 non-null    float64
10  1500m                  41 non-null    float64
11  Rank                   41 non-null    int64
12  Points                 41 non-null    int64
13  Competition            41 non-null    object
dtypes: float64(10), int64(2), object(2)
```

La commande `df.info()` fournit un aperçu concis du `DataFrame` `df` dans `Pandas`, révélant qu'il contient 41 lignes, numérotées de 0 à 40, et 14 colonnes. Ce résumé indique pour chaque colonne son nom, le nombre de valeurs non nulles (41 pour toutes les colonnes dans ce cas), ainsi que leur type de données. Parmi les colonnes, 10 sont de type `float64` (liées aux performances), 2 sont de type `int64` (concernant le rang et les points), et 2 autres sont de type `object` (représentant les noms des athlètes et la compétition).

Cette analyse initiale des types de données et du nombre de valeurs non nulles permet également de vérifier l'intégrité des données avant de procéder à des analyses plus approfondies. En confirmant que toutes les colonnes contiennent des valeurs valides et qu'il n'y a pas de valeurs manquantes, nous nous assurons que les résultats des analyses ultérieures seront fiables. De plus, cette étape nous aide à identifier les variables pertinentes pour les modèles prédictifs, en mettant en lumière celles qui nécessitent peut-être des transformations ou des normalisations. Par exemple, les colonnes de type `float64` pourraient être standardisées pour garantir une contribution équitable à l'Analyse en Composantes Principales (ACP). Ainsi, cette première exploration des données établit une base solide pour le développement de modèles analytiques robustes et précis.

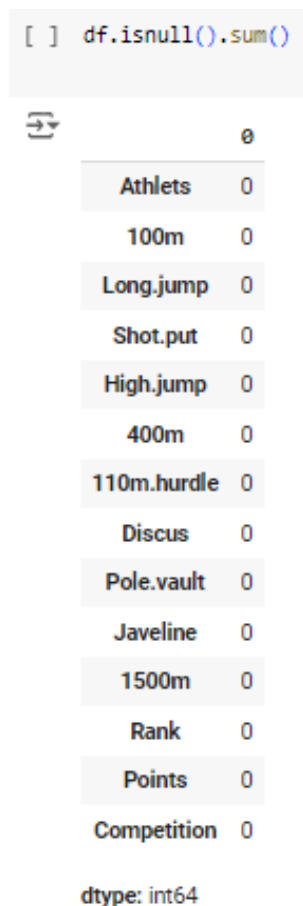
```
[ ] df.describe()
```

```
&#x2191;
```

	100m	Long_jump	Shot_put	High_jump	400m	110m.hurdle	Discus	Pole.vault	Javeline	1500m	Rank	Points
count	41.000000	41.000000	41.000000	41.000000	41.000000	41.000000	41.000000	41.000000	41.000000	41.000000	41.000000	41.000000
mean	10.998049	7.260000	14.477073	1.976829	49.616341	14.605854	44.325610	4.762439	58.316585	279.024878	12.121951	8005.365854
std	0.263023	0.316402	0.824428	0.088951	1.153451	0.471789	3.377845	0.278000	4.826820	11.673247	7.918949	342.385145
min	10.440000	6.610000	12.680000	1.850000	46.810000	13.970000	37.920000	4.200000	50.310000	262.100000	1.000000	7313.000000
25%	10.850000	7.030000	13.880000	1.920000	48.930000	14.210000	41.900000	4.500000	55.270000	271.020000	6.000000	7802.000000
50%	10.980000	7.300000	14.570000	1.950000	49.400000	14.480000	44.410000	4.800000	58.360000	278.050000	11.000000	8021.000000
75%	11.140000	7.480000	14.970000	2.040000	50.300000	14.980000	46.070000	4.920000	60.890000	285.100000	18.000000	8122.000000
max	11.640000	7.960000	16.360000	2.150000	53.200000	15.670000	51.650000	5.400000	70.520000	317.000000	28.000000	8893.000000

La commande `df.describe()` produit un résumé statistique des colonnes numériques du DataFrame `df`, fournissant des informations essentielles sur la distribution des données. Le résultat indique que chaque épreuve, comme le 100 m, le saut en longueur, le lancer de poids, etc., comprend 41 observations. Des statistiques importantes sont présentées, notamment la moyenne, l'écart-type, ainsi que les valeurs minimales et maximales.

```
[ ] df.isnull().sum()
```



Athlets	0
100m	0
Long.jump	0
Shot.put	0
High.jump	0
400m	0
110m.hurdle	0
Discus	0
Pole.vault	0
Javeline	0
1500m	0
Rank	0
Points	0
Competition	0

dtype: int64

La commande `df.isnull().sum()` permet d'évaluer la présence de valeurs manquantes dans chaque colonne du DataFrame `df`. Les résultats indiquent qu'aucune des colonnes, y compris Athletes, 100m, Long.jump, Shot.put, High.jump, 400m, 110m.hurdle, Discus, Pole.vault, Javelin, 1500m, Rank, Points et Competition, ne contient de valeurs manquantes. Cela assure l'intégrité des analyses à venir, en évitant que des données manquantes n'influencent les résultats. En résumé, l'absence de valeurs nulles confirme que les données sont complètes et prêtes pour des analyses statistiques ou des modèles prédictifs.

4 Standardisation :

Avant d'effectuer une Analyse en Composantes Principales (ACP), il est essentiel de centrer et de normaliser les variables quantitatives pour garantir une contribution équilibrée de chaque variable à l'analyse. Étant donné que les performances des athlètes dans diverses disciplines sont souvent évaluées sur des échelles différentes, l'absence de standardisation peut entraîner une domination des variables à valeurs élevées dans l'ACP. La standardisation recentre les données autour de zéro et les ajuste à une échelle de variance

unitaire, ce qui permet une interprétation appropriée des composantes principales. Par exemple, la première composante principale (PC1) peut expliquer 32,7 % de la variance, soulignant l'importance de cette étape pour obtenir des résultats précis et exploitables.

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

performance_columns = ['100m', 'Long.jump', 'Shot.put', 'High.jump', '400m', '110m.hurdle', 'Discus', 'Pole.vault', 'Javeline', '1500m']

scaler = StandardScaler()
data_scaled = scaler.fit_transform(data[performance_columns])

pca = PCA()
pca_result = pca.fit_transform(data_scaled)

explained_variance = pca.explained_variance_ratio_

pca_df = pd.DataFrame(pca_result, columns=[f'PC{i+1}' for i in range(pca_result.shape[1])])

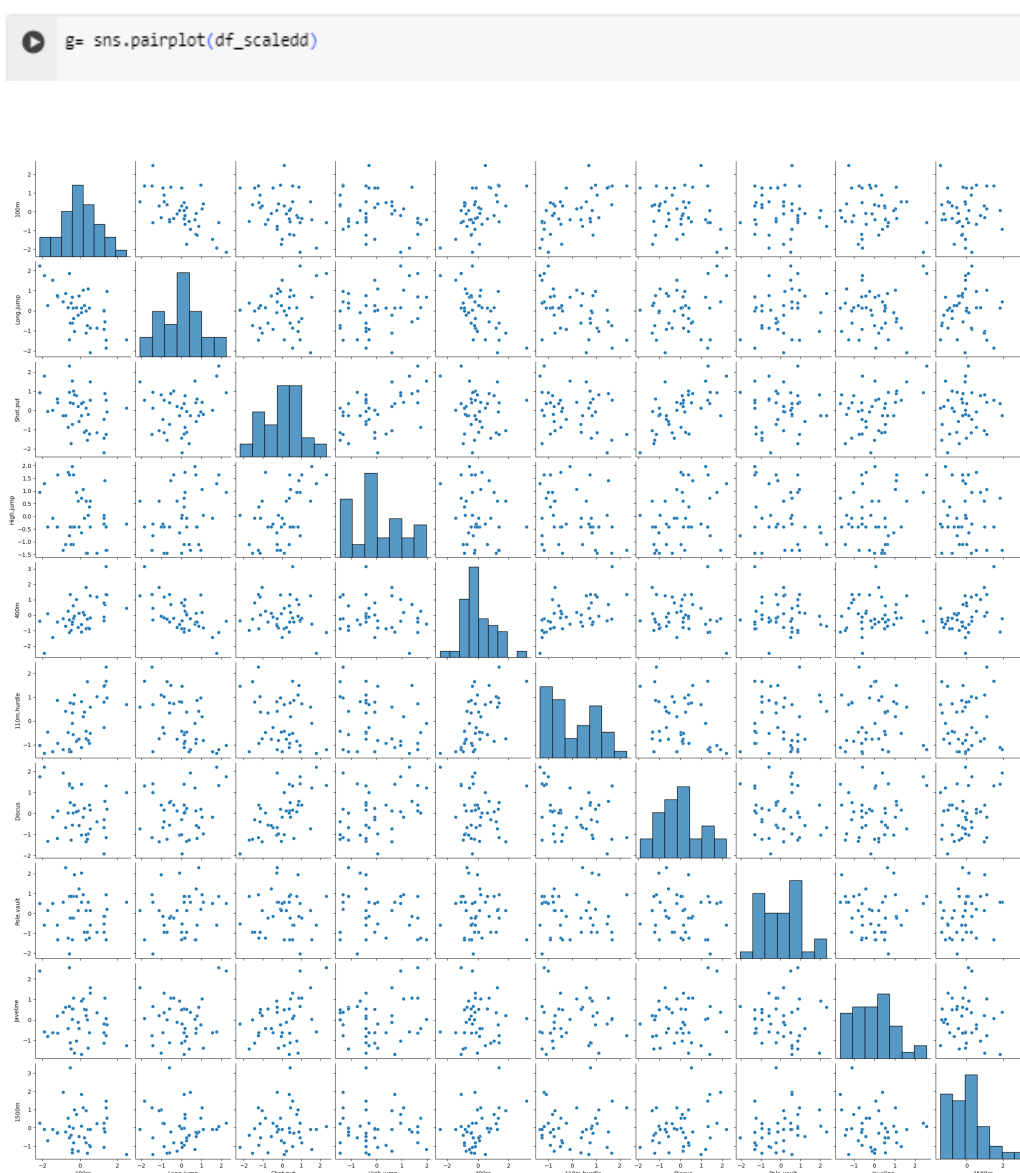
pca_df['Athlets'] = data['Athlets']
pca_df['Rank'] = data['Rank']
pca_df['Points'] = data['Points']
pca_df['Competition'] = data['Competition']

print("Variance expliquée par chaque composante principale :", explained_variance)
print(pca_df.head())
```

```
Variance expliquée par chaque composante principale : [0.32719055 0.1737131 0.14049167 0.10568504 0.06847735 0.05992687
0.04512353 0.03968766 0.02148149 0.01822275]
   PC1    PC2    PC3    PC4    PC5    PC6    PC7 \
0 -0.791628 0.771611 0.826841 1.174627 0.707159 1.030620 0.551523
1 -1.234991 0.574578 2.141247 -0.354845 -1.974571 -0.690126 0.707974
2 -1.358215 0.484021 1.956258 -1.856524 0.795215 -0.732751 0.189939
3 0.609515 -0.874629 0.889941 2.220612 0.361636 -0.275598 -0.049611
4 0.585968 2.130954 -1.225157 0.873579 1.251369 0.104606 0.573925

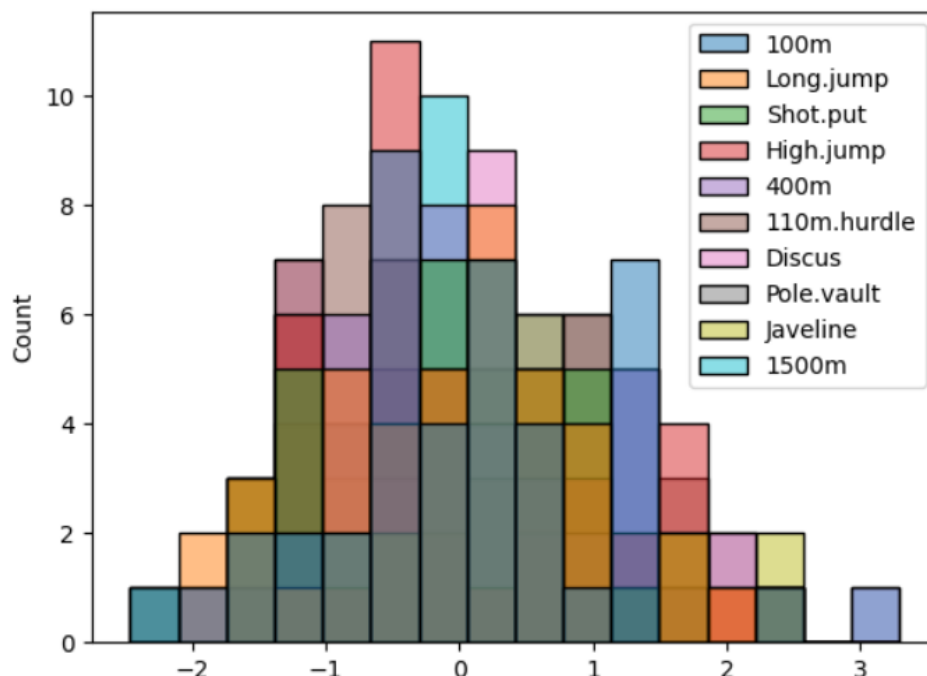
   PC8    PC9    PC10 Athlets Rank Points Competition
0 -0.435655 -0.137559 -0.500774 SEBRLE 1 8217 Decastar
1 -0.603419 -0.649244 0.266119 CLAY 2 8122 Decastar
2 -0.250297 -0.800654 -0.523269 KARPOV 3 8099 Decastar
3 0.067458 -0.723281 -0.188459 BERNARD 4 8067 Decastar
4 0.094604 -0.202216 -0.056443 YURKOV 5 8036 Decastar
```

Effectuer une Analyse en Composantes Principales (ACP) sur les dix colonnes représentant les performances des athlètes dans les épreuves du décathlon permet de réduire la dimensionnalité des données tout en préservant l'essentiel de la variance. Cela facilite l'analyse des corrélations entre les différentes épreuves et l'identification des facteurs clés influençant les performances. Les variables non incluses dans l'ACP, telles que le rank, les points et la compétition, sont également cruciales pour interpréter les résultats. Le rank permet d'explorer la relation entre les composantes principales et le classement des athlètes, tandis que les points valident les regroupements selon les scores totaux. Enfin, la variable compétition est essentielle pour comparer les performances à travers divers événements, tels que les Jeux Olympiques et le Décastar, enrichissant ainsi l'analyse globale.

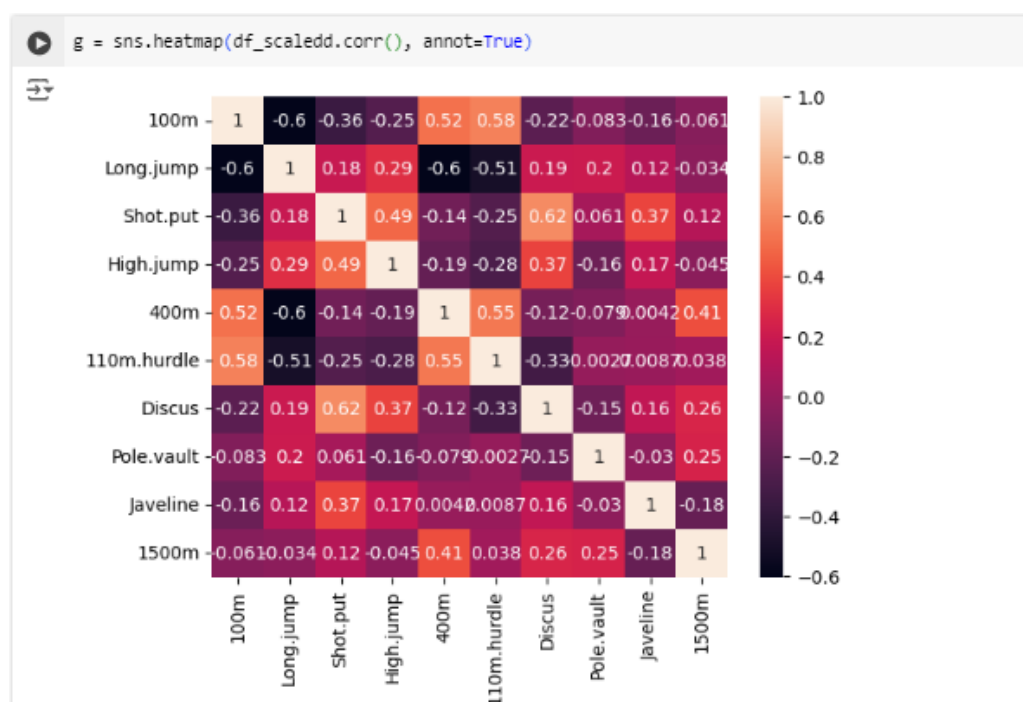


Ce diagramme de dispersion offre une analyse approfondie des relations entre plusieurs variables quantitatives, en examinant à la fois leur distribution individuelle et leurs corrélations. Les histogrammes présents sur la diagonale illustrent la répartition de chaque variable, fournissant des informations sur leur asymétrie ou leur étalement. Dans les autres cases, les nuages de points montrent les relations entre chaque paire de variables : une tendance linéaire signale une corrélation positive ou négative, tandis qu'une dispersion aléatoire indique l'absence de corrélation. Analyser ces interactions permet de repérer des corrélations fortes ou faibles, ainsi que d'identifier des regroupements ou des anomalies au sein des données.

```
g = sns . histplot ( df_scaled )
```



Ce graphique affiche un histogramme empilé qui démontre la répartition des performances à travers plusieurs disciplines sportives, chaque couleur correspondant à une épreuve spécifique. La forme générale de la distribution se rapproche d'une courbe normale, indiquant une concentration des performances autour de la moyenne, avec une diminution progressive vers les valeurs extrêmes. Cela suggère que la plupart des disciplines présentent des performances moyennes comparables, bien que certaines se démarquent par des résultats plus exceptionnels. L'historgramme met en lumière les disciplines les plus fréquentes dans certaines plages de valeurs, révélant ainsi une répartition globalement équilibrée, tout en montrant des variations particulières selon les épreuves.

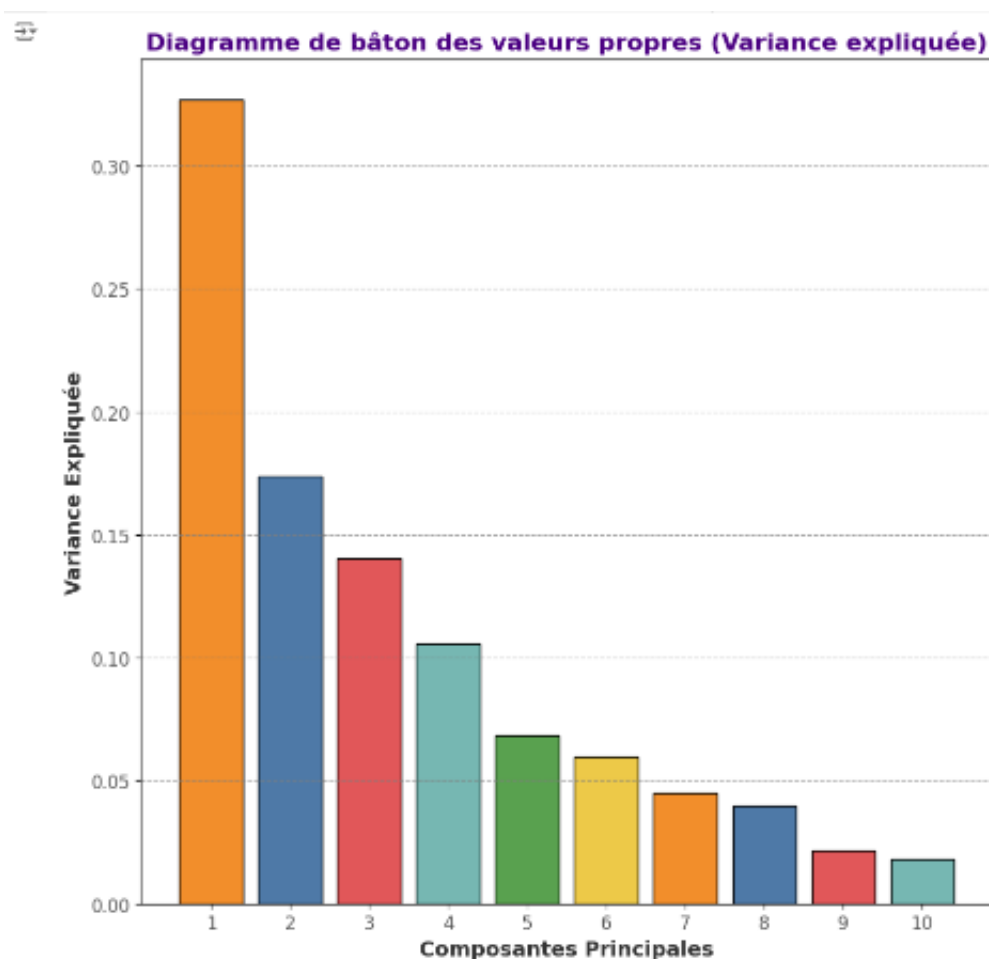


Cette figure présente une matrice de corrélation sous forme de carte thermique (heatmap) entre différentes épreuves d'athlétisme, telles que le 100 m, le saut en longueur, et le lancer du poids. Les valeurs de corrélation, comprises entre -1 et 1, sont visualisées par un dégradé de couleurs allant du bleu foncé (fortes corrélations négatives) au rouge foncé (fortes corrélations positives). Les corrélations positives indiquent que de bonnes performances dans une épreuve sont souvent associées à de bonnes performances dans une autre (par exemple, une corrélation de 0,62 entre le lancer du poids et le lancer du disque). À l'inverse, les corrélations négatives révèlent des relations inverses entre les performances (par exemple, une corrélation de -0,36 entre le 100 m et le lancer du poids). Cette visualisation permet de mieux comprendre les relations entre les disciplines et d'identifier comment elles se complètent ou s'opposent chez les athlètes polyvalents.

5 Les valeurs propres :

```
[39] from sklearn.decomposition import PCA

[38] import matplotlib.pyplot as plt
import numpy as np
colors = ['#f28e2b', '#4e79a7', '#e15759', '#76b7b2', '#59a14f', '#edc948']
components = np.arange(1, len(explained_variance) + 1)
plt.figure(figsize=(9, 9))
plt.bar(components, explained_variance, color=colors[:len(explained_variance)], edgecolor='black')
plt.xlabel('Composantes Principales', fontsize=14, fontweight='bold', color='black')
plt.ylabel('Variance Expliquée', fontsize=14, fontweight='bold', color='black')
plt.title('Diagramme de bâton des valeurs propres (Variance expliquée)', fontsize=16, fontweight='bold', color='black')
plt.xticks(components, fontsize=12, color='black')
plt.yticks(fontsize=12, color='black')
plt.grid(axis='y', linestyle='--', color='gray', alpha=0.7)
plt.tight_layout()
plt.show()
```



Le diagramme à barres illustre la distribution des valeurs propres liées aux composantes principales (PC), mettant en évidence la proportion de variance expliquée par chacune. La première composante principale (PC1) capture la plus grande part de la variance, comme l'indique la barre la plus élevée à gauche, suivie des autres composantes (PC2, PC3, etc.), qui expliquent de moins en moins de variance. Si PC1 représente une part significative de la variance (par exemple, plus de 30 %), cela indique que cette seule composante peut condenser une grande partie des informations initiales. Bien que les composantes suivantes expliquent moins de variance, elles demeurent essentielles pour comprendre les subtilités des données. Ce type de graphique est couramment utilisé pour optimiser la réduction de la dimensionnalité en sélectionnant le nombre adéquat de composantes principales à conserver, généralement dans le but d'atteindre 80-90 % de variance expliquée, ce qui simplifie l'analyse tout en conservant l'essentiel des informations.

6 Détermination des axes à conserver pour le k-means sur les performances des athlètes en dimension réduite

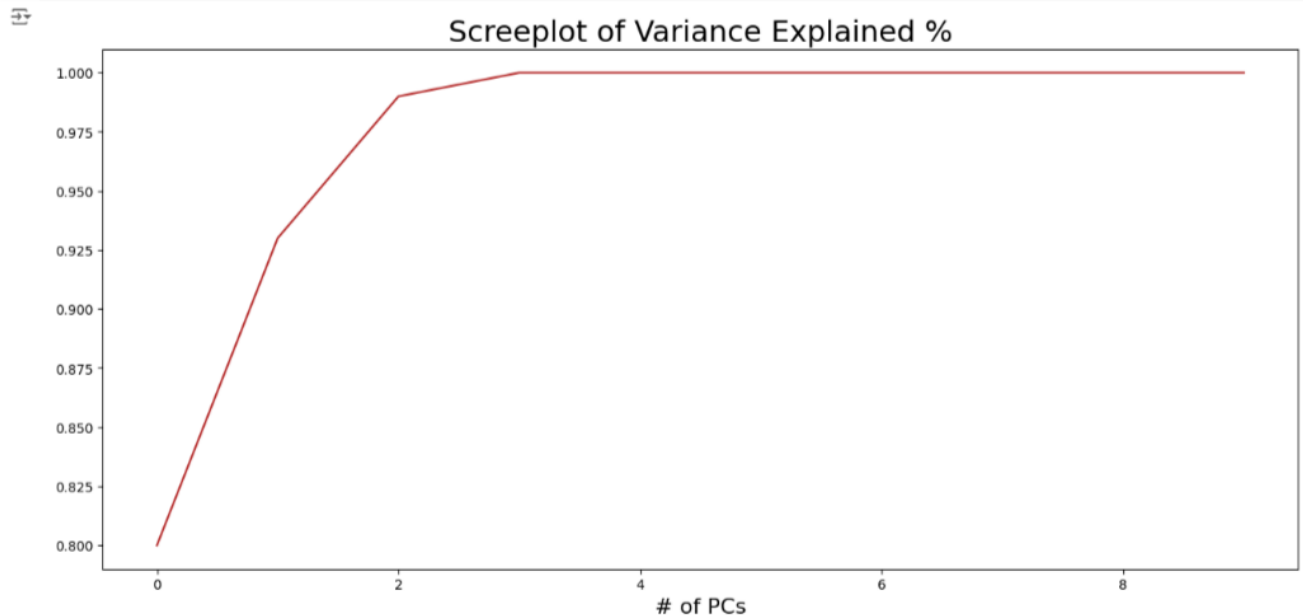
```
[ ] pca = PCA()
    df_pca = pca.fit_transform(df)
    df_pca = pd.DataFrame(df_pca)
    print(df_pca.shape)
    df_pca.head()
```

(41, 10)

	0	1	2	3	4	5	6	7	8	9
0	12.126899	5.319245	-3.011589	-0.744027	0.089672	0.121969	0.152492	0.071655	0.230765	0.068469
1	22.634006	4.818603	3.470849	-0.729689	-1.312264	-0.134853	0.063602	-0.206027	-0.147594	-0.085756
2	22.016009	-5.126563	4.727482	-1.501424	-0.181608	0.213268	-0.152025	-0.316580	0.133563	0.055355
3	0.389697	3.376956	-4.530399	-1.180183	-0.067118	0.558886	0.203180	-0.281749	-0.069411	0.017973
4	-2.856746	5.324504	0.644554	1.164329	0.291117	0.541054	0.029983	0.013778	0.155482	0.093700

Le code réalise une Analyse en Composantes Principales (ACP) sur le DataFrame `df` en utilisant l'objet `PCA()` pour transformer les données initiales en un nouveau DataFrame nommé `df_pca`. La méthode `fit_transform()` ajuste le modèle d'ACP aux données et projette celles-ci dans un nouvel espace de composantes principales, produisant un DataFrame de dimensions (41, 10), ce qui signifie qu'il comporte 41 observations (athlètes) et 10 composantes principales. Chaque colonne de `df_pca` correspond à une composante principale, tandis que chaque ligne représente les nouvelles coordonnées des athlètes dans cet espace réduit, capturant l'essentiel de la variance des performances. L'affichage des cinq premières lignes permet de visualiser les valeurs projetées des athlètes sur les 10 composantes principales, facilitant ainsi l'analyse et la représentation des données dans cet espace de dimension réduite tout en préservant les informations essentielles concernant les performances.

```
[ ] variance_exp_cumsum = pca.explained_variance_ratio_.cumsum().round(2)
fig, axes = plt.subplots(1,1,figsize=(16,7), dpi=100)
plt.plot(variance_exp_cumsum, color='firebrick')
plt.title('Screeplot of Variance Explained %', fontsize=22)
plt.xlabel('# of PCs', fontsize=16)
plt.show()
```



Le graphique présenté est un scree plot qui montre la variance expliquée cumulée en fonction du nombre de composantes principales (PCs). Ce type de graphique permet d'évaluer l'efficacité de chaque composante dans la représentation des données d'origine. On constate une forte augmentation initiale de la variance expliquée, suivie d'une rapide convergence vers 100 %. Cela indique que les premières composantes principales (PC1 et PC2) capturent la majorité de la variance des données, tandis que les composantes suivantes sont beaucoup moins pertinentes. Cette convergence suggère que seules les premières composantes suffisent à saisir presque toute la variance, ce qui facilite la réduction de la dimensionnalité des données sans perte significative d'information.

7 Représentation graphique des individus et des sports :

```
[ ] import matplotlib.pyplot as plt
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Select performance columns
performance_columns = ['100m', 'Long.jump', 'Shot.put', 'High.jump', '400m', '110m.hurdle',
                      'Discus', 'Pole.vault', 'Javeline', '1500m']

# Standardizing the data
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data[performance_columns])

# Perform PCA
pca = PCA(n_components=2)
pca_result = pca.fit_transform(data_scaled)

plt.figure(figsize=(10, 6))

# Modify athlete points color and marker size
plt.scatter(pca_result[:, 0], pca_result[:, 1], color='orange', s=100, label='Athletes', alpha=0.7)

# Add labels for athletes
for i, athlete in enumerate(data['Athletes']):
    plt.text(pca_result[i, 0], pca_result[i, 1], athlete, fontsize=9, color='black')

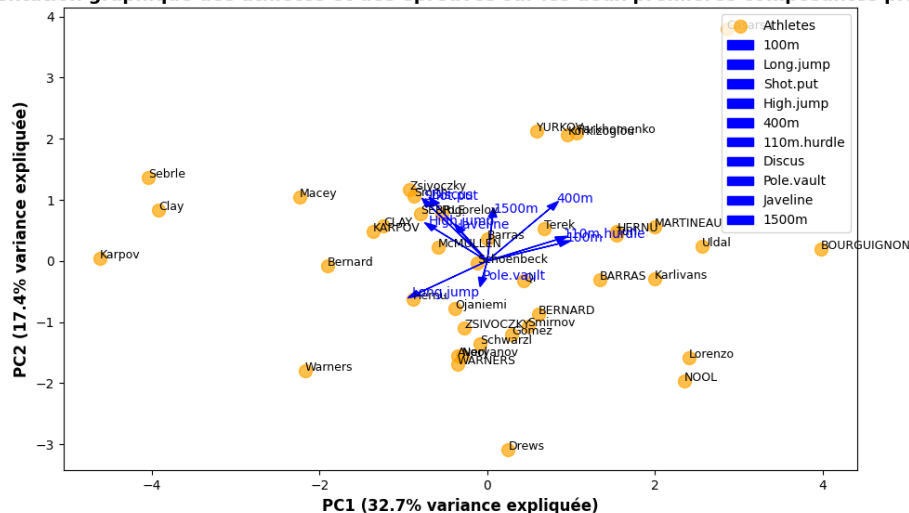
# Plot arrows for sports (representing the variables)
pca_components = pca.components_.T # Get the directions of the sports
for i, sport in enumerate(performance_columns):
    plt.arrow(0, 0, pca_components[i, 0]*2, pca_components[i, 1]*2,
             color='blue', head_width=0.1, label=sport)
    plt.text(pca_components[i, 0]*2.2, pca_components[i, 1]*2.2, sport, color='blue', fontsize=10)

# Add labels and title
plt.xlabel('PC1 (32.7% variance expliquée)', fontsize=12, fontweight='bold')
plt.ylabel('PC2 (17.4% variance expliquée)', fontsize=12, fontweight='bold')
plt.title('Représentation graphique des athlètes et des épreuves sur les deux premières composantes principales', fontsize=14, fontweight='bold')

# Add legend for athletes
plt.legend(loc='upper right')

# Show plot
plt.tight_layout()
plt.show()
```

Représentation graphique des athlètes et des épreuves sur les deux premières composantes principales



Pour représenter les athlètes et les disciplines sportives sur les deux premières dimensions, nous commençons par tracer les athlètes sur ces composantes principales à l'aide de la fonction `plt.scatter`. Ensuite, nous utilisons la fonction `plt.arrow` pour illustrer les disciplines sportives avec des flèches, indiquant ainsi la direction des variables sur ces dimensions. En analysant cette représentation, nous pouvons identifier différents groupes d'athlètes. Par exemple, ceux ayant une première composante principale (PC1) inférieure à 0 semblent se démarquer dans les épreuves de course, tandis que ceux avec une valeur de PC1 supérieure à 0 excellent davantage dans les épreuves de lancer ou de saut.

8 Travail en autonomie :

```
[ ] import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))

# Scatter plot to show athletes by their rank and points, colored by competition
sns.scatterplot(data=data, x='Points', y='Rank', hue='Competition', palette='coolwarm', s=150, marker='o', edgecolor='black', alpha=0.7)

# Add labels and title with custom font sizes and bold font weight
plt.xlabel('Points', fontsize=14, fontweight='bold', color='darkblue')
plt.ylabel('Rank', fontsize=14, fontweight='bold', color='darkblue')
plt.title('Visualisation des Athlètes par Points, Rang et Compétition', fontsize=16, fontweight='bold', color='purple')

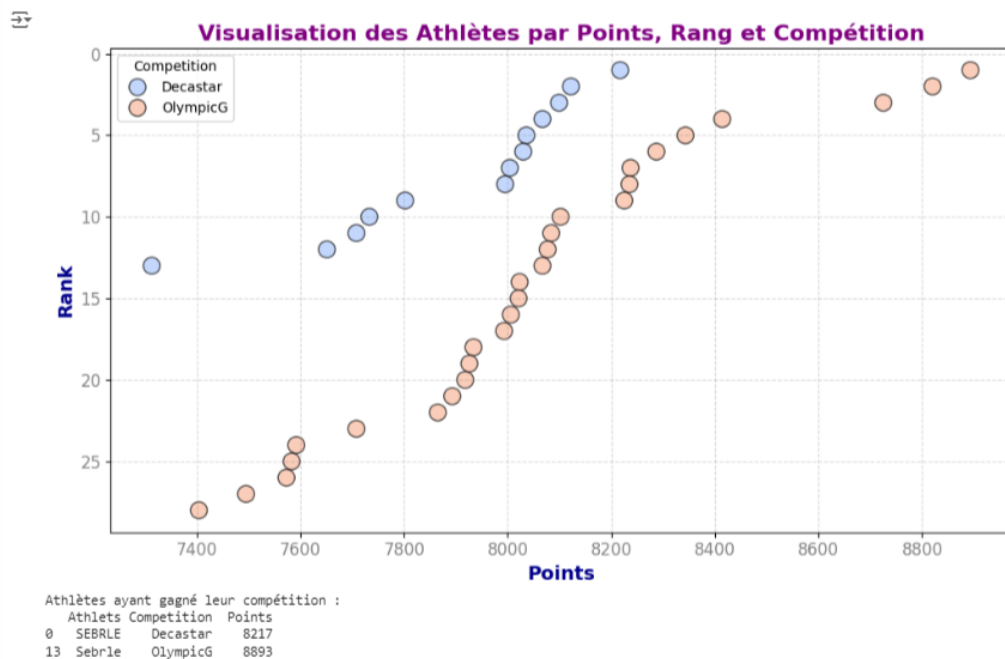
# Invert y-axis to make the 1st rank appear on top
plt.gca().invert_yaxis()

# Customize ticks for better readability
plt.xticks(fontsize=12, color='gray')
plt.yticks(fontsize=12, color='gray')

# Add gridlines for better visualization
plt.grid(True, which='both', axis='both', linestyle='--', color='gray', alpha=0.3)

# Show plot with tighter layout
plt.tight_layout()
plt.show()

# Identify winning profiles (Rank = 1 in each competition)
winners = data[data['Rank'] == 1]
print("Athlètes ayant gagné leur compétition :)")
print(winners[['Athlètes', 'Compétition', 'Points']])
```



Le graphique présente une visualisation des athlètes selon leur rang et leurs points, en distinguant clairement deux compétitions : le Decastar et l'OlympicG. Les athlètes participant au Decastar sont représentés en bleu, tandis que ceux du OlympicG sont en orange. On observe une corrélation nette entre le nombre de points accumulés par un athlète et son rang, où des points plus élevés correspondent à un meilleur rang (1 étant le meilleur). De plus, les participants du Decastar affichent des points moyens légèrement inférieurs à ceux du OlympicG, dont les meilleurs athlètes dépassent souvent 8800 points. Les gagnants de chaque compétition sont également mis en évidence : Sebrle a remporté le Decastar avec un score de 8217 points, tout en triomphant également au OlympicG

avec un impressionnant total de 8893 points. Ce résultat souligne la supériorité de cet athlète dans les deux événements.

9 Conclusion :

L'Analyse en Composantes Principales (ACP) a efficacement réduit la dimensionnalité des données relatives aux performances des athlètes, tout en conservant l'essentiel des informations. Les premières composantes principales ont enregistré la majorité de la variance, ce qui a facilité l'analyse des performances dans un espace plus compact. Cette méthode a mis en évidence des corrélations entre certaines épreuves et a permis de différencier les profils des athlètes selon leurs forces, notamment dans les disciplines de course ou de lancer. De plus, l'ACP peut être utilisée pour optimiser des modèles prédictifs, effectuer des regroupements d'athlètes ou intégrer des variables supplémentaires telles que l'âge ou les conditions environnementales. En outre, cette approche pourrait être étendue à d'autres disciplines sportives pour identifier des tendances de performance et mieux comprendre l'impact des compétences multidimensionnelles. Ainsi, l'ACP constitue un outil puissant et polyvalent pour l'analyse et la prévision dans le domaine du sport.